

Open in app ↗

 Search Write

Wavelets + LSTM — Predict Stock Price with Python



Simon Leung · 8 min read · Nov 1, 2024



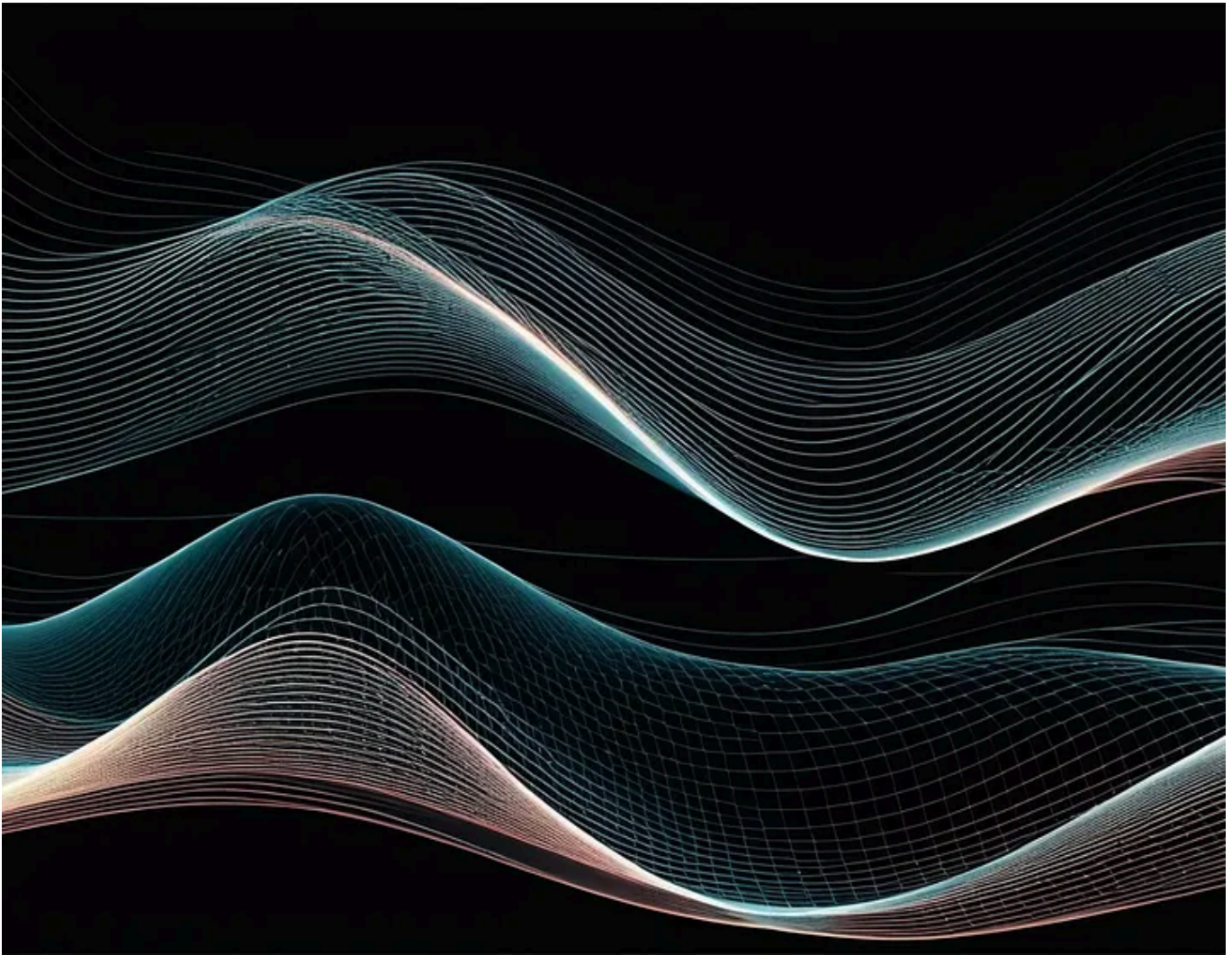
125



2



Using Wavelet Transform with LSTM model



Created by deepdreamgenerator.com

What is Wavelet? The word “wavelet” means a “small wave”.

Wavelet analysis serves as a mathematical tool applicable across various research fields. Recently, wavelets have been prominently utilized to analyze seismic signal, climate system (El Niño-Southern Oscillation, ENSO), medical diagnostics (ECG), data compression (fingerprints compression), and image denoising, etc. Wavelet transform decomposes a signal into different components, each representing a different frequency band and localized in time. Imagine breaking down a piece of music into individual notes or segments that can be analyzed separately — wavelet transform does something similar for complex signals, providing a clearer picture of the

underlying patterns. The process of converting the decomposed signal back to its original wave form is referred to as inverse wavelet transform.

Compared with Fourier Transform:

Fourier Transform (FT):

- Decomposes a signal into sinusoids (sine and cosine waves) of varying frequencies.
- Loses time information, providing only frequency information for the entire signal.
- Works best with stationary signals, where frequencies do not change over time.

Wavelet Transform (WT):

- Decomposes a signal into wavelets (short, localized waves).
- Provides both time and frequency information, enabling better analysis of non-stationary signals (signals where frequency changes over time).
- Ideal for signals with abrupt changes or localized features.

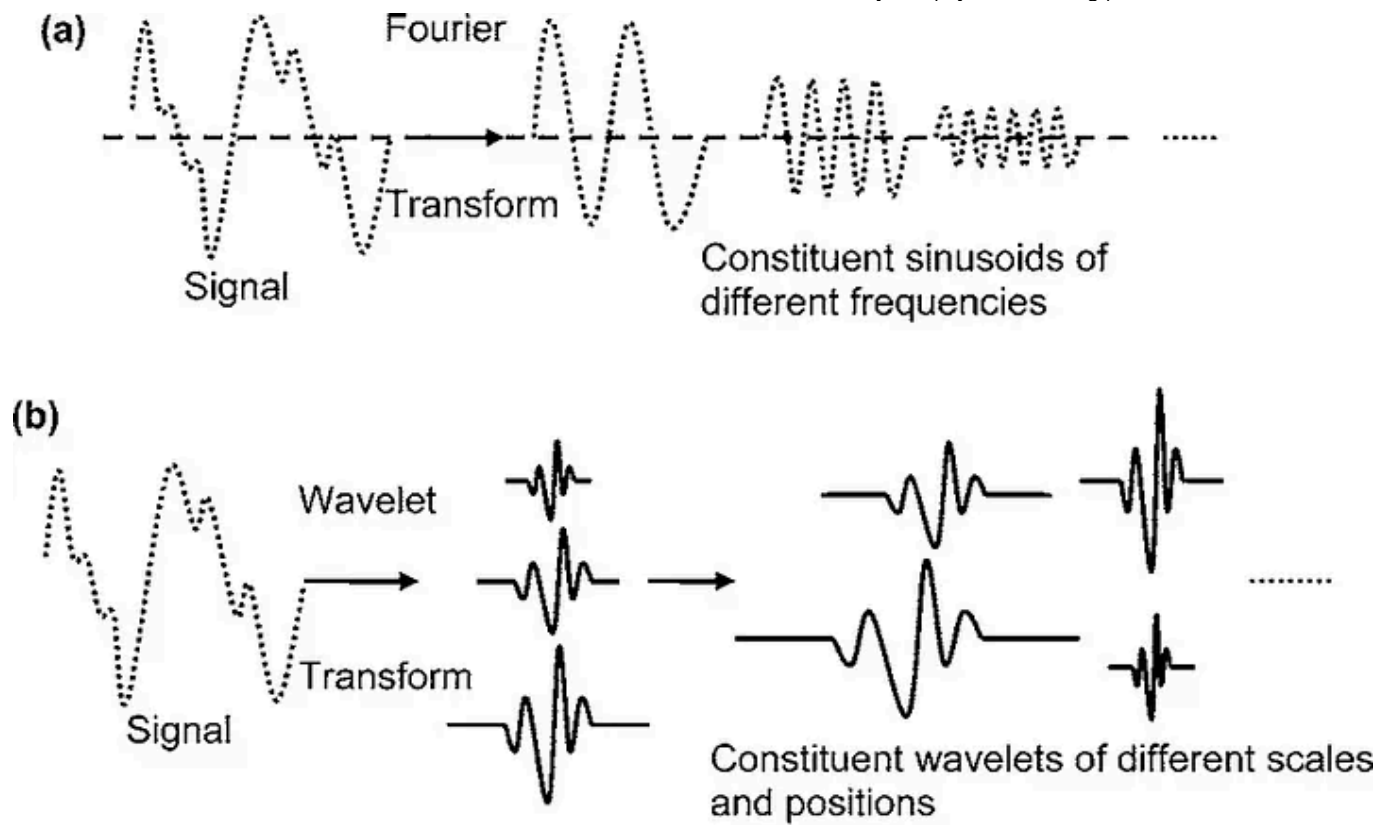
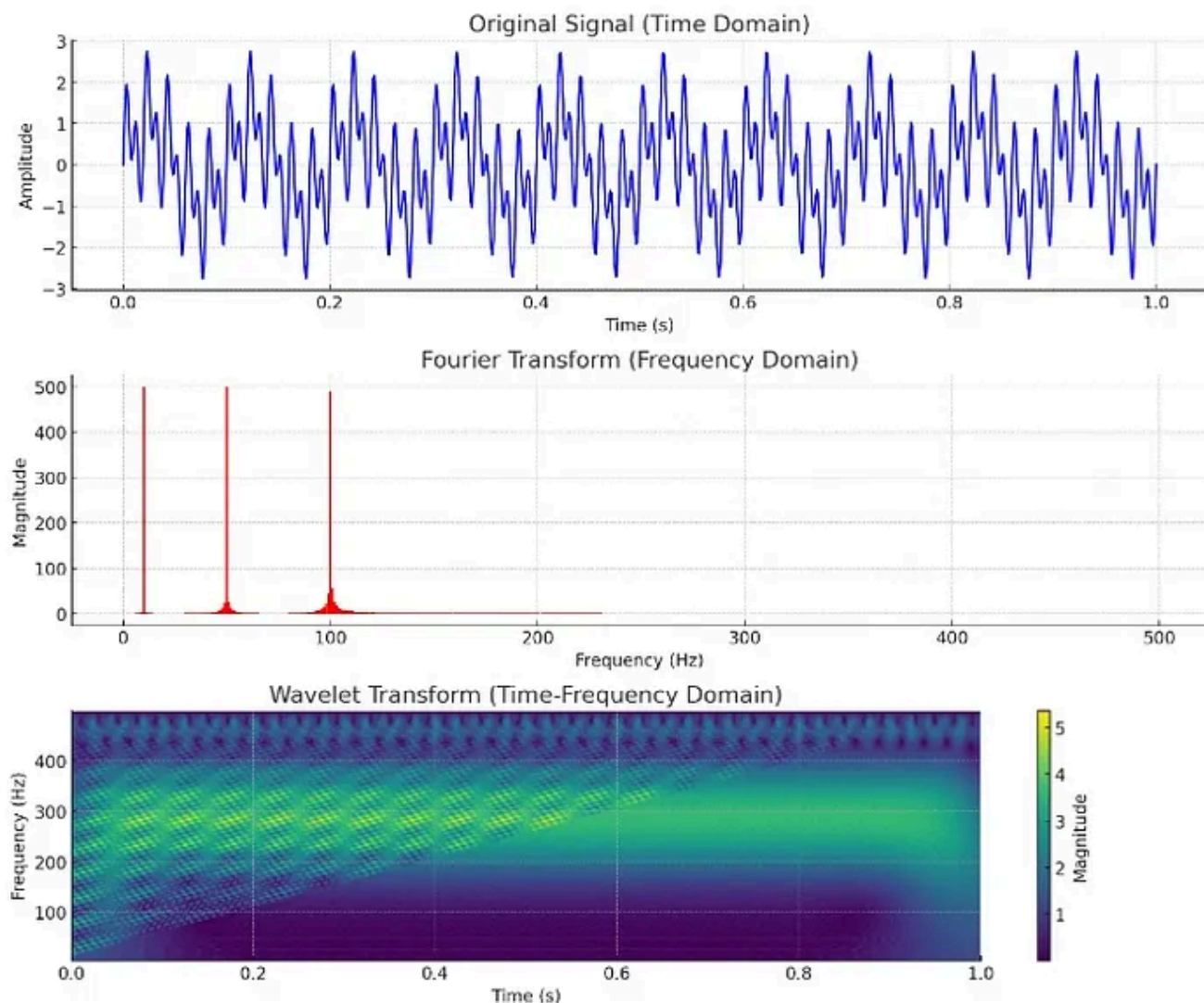


Image from [1]



Additional articles on Wavelet:

[Basic Concepts of Wavelets Transform](#)

[What is Wavelet Transform? An Introduction and Example](#)

[What, Exactly, Are Wavelets?](#)

[Fourier vs. Wavelet Transform: What's the Difference?](#)

[What is Wavelet and How We Use It for Data Science](#)

[Wavelet Transform: A Practical Approach to Time-Frequency Analysis](#)

[Signal Extension Modes for Discrete Wavelet Transform in PyWavelets](#)

[2D-DWT: A brief intro](#)

Disclaimer:

The information presented herein is provided solely for informational purposes and should not be construed as financial or investment advice. Any investment decisions or actions taken based on this information are made at your own discretion and risk. Stock prices and investment values are subject to fluctuation due to various market factors and conditions.

I strongly recommend consulting with a qualified financial professional for personalized guidance tailored to your individual financial situation and objectives before making any investment decisions.

Let's start applying this to stock price time series data.

[\[Full source code here\]](#)

Load “Apple Inc.” stock data.

```
import pandas as pd
import matplotlib.pyplot as plt
import yfinance as yf

# Download historical stock data
ticker_symbol = "AAPL" # Example: Apple Inc.
stock_data = yf.download(ticker, start='2010-01-01', end='2024-10-01')
```

Test for Stationarity

```
# Augmented Dickey-Fuller test for stationarity
result = adfuller(stock_data['Close'], regression = 'ctt')
print('ADF Statistic: %f' % result[0])
print('p-value: %f' % result[1])
print('Critical Values:')
for key, value in result[4].items():
    print('\t%s: %.3f' % (key, value))
if result[1] > 0.05 :
    print('Fail to reject the null hypothesis (H0), the data has a unit root and is non-stationary')
elif result[1] <= 0.05 :
    print('Reject the null hypothesis (H0), the data does not have a unit root and is stationary')
```

```
ADF Statistic: -3.534705
p-value: 0.104391
Critical Values:
1%: -4.374
5%: -3.834
10%: -3.554
Fail to reject the null hypothesis (H0), the data has a unit root and is non-stationary
```

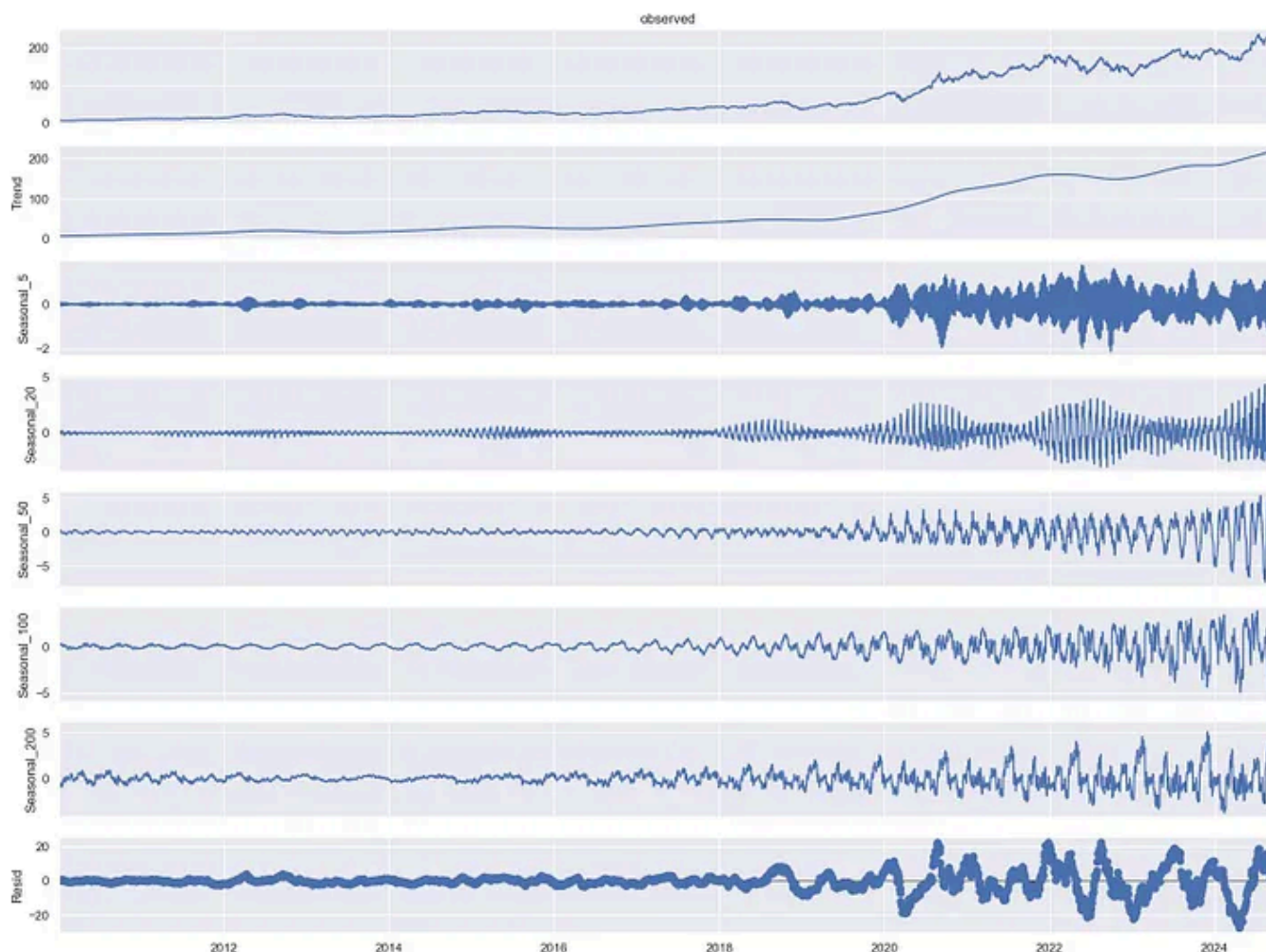
Multiple Seasonal-Trend decomposition using LOESS (MSTL)

```
from statsmodels.tsa.seasonal import MSTL

plt.rc("figure", figsize=(16, 12))
plt.figure(figsize=(15,10))
# decomposed by 5,20,50,100,200-days
MSTL_result = MSTL(endog=stock_data['Close'], periods=(5, 20, 50, 100, 200)).fit()

MSTL_result.plot()

plt.show()
```

From the above decomposition plot, it exhibits cyclic waveforms after the seasonal 5-day period. Even the residuals still show significant autocorrelation.

```
import pandas as pd
import numpy as np
from statsmodels.stats.diagnostic import acorr_ljungbox

def ljung_box_test_original(data, lags_list=[5, 10]):
    # Perform the Ljung-Box test for both lag=5 and lag=10
    for lag in lags_list:
        lb_test = acorr_ljungbox(data, lags=lags_list, return_df=True)

        # Extract p-value
        p_value = lb_test['lb_pvalue'].iloc[0]

        # Interpret the result
```



```

if p_value > 0.05:
    interpretation = f"Fail to reject the null hypothesis at lag {lag}:
else:
    interpretation = f"Reject the null hypothesis at lag {lag}: Signific

print(f"Ljung-Box test p-value for lag {lag}: {p_value:.5f}")
print(f"Interpretation: {interpretation}")
print

ljung_box_test_original(MSTL_result.resid, [10, 15])

```

```

Ljung-Box test p-value for lag 10: 0.00000
Interpretation: Reject the null hypothesis at lag 10: Significant autocorrelatio
Ljung-Box test p-value for lag 15: 0.00000
Interpretation: Reject the null hypothesis at lag 15: Significant autocorrelatio

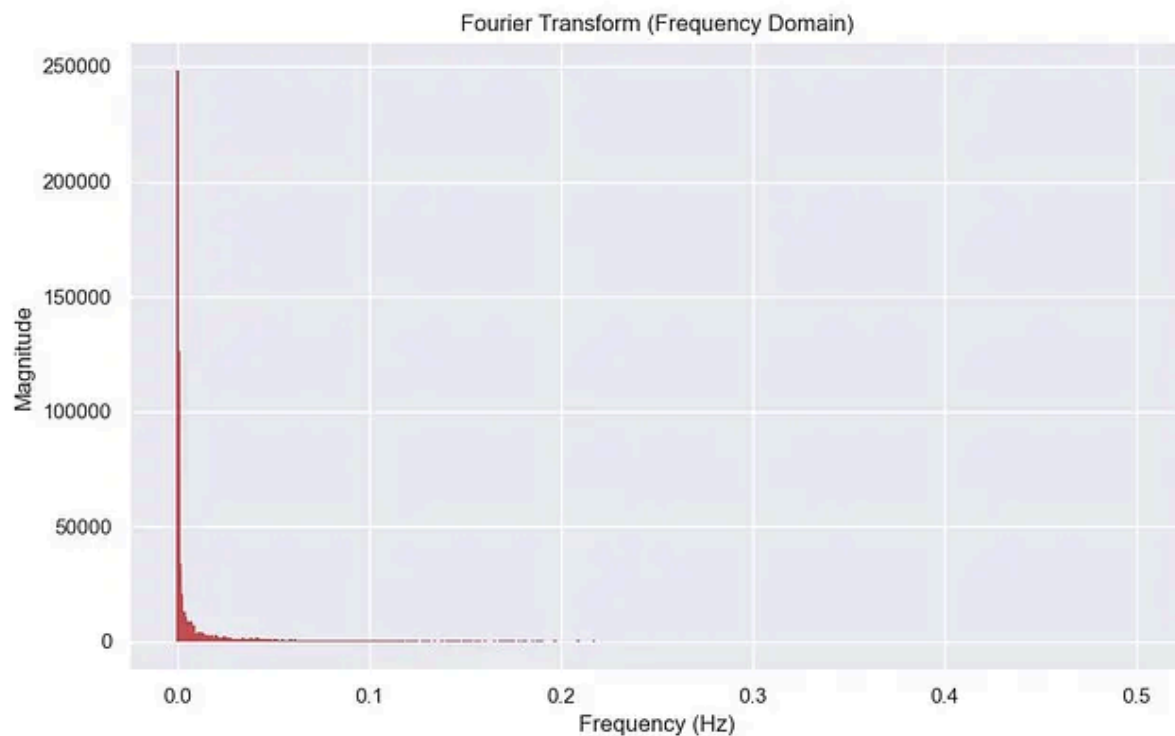
```

Try Fourier Transform, but no visible frequency break-down.

```

# Check seasonal cycle using Spectral Analysis (Fourier Transform)
freqs = np.fft.fftfreq(len(stock_data['Close']))
magn = np.abs(np.fft.fft(stock_data['Close']))
plt.figure(figsize=(10, 6))
plt.stem(freqs[:len(freqs)//2],
         magn[:len(magn)//2],
         linefmt='r', markerfmt=" ", basefmt=" ")
plt.title("Fourier Transform (Frequency Domain)")
plt.xlabel("Frequency (Hz)")
plt.ylabel("Magnitude")
plt.show()

```



Now, install PyWavelets library, and try Wavelet Transform.

```
# https://pywavelets.readthedocs.io/en/latest/install.html
pip install PyWavelets
# OR
conda install -c conda-forge pywavelets
```

```
import pywt

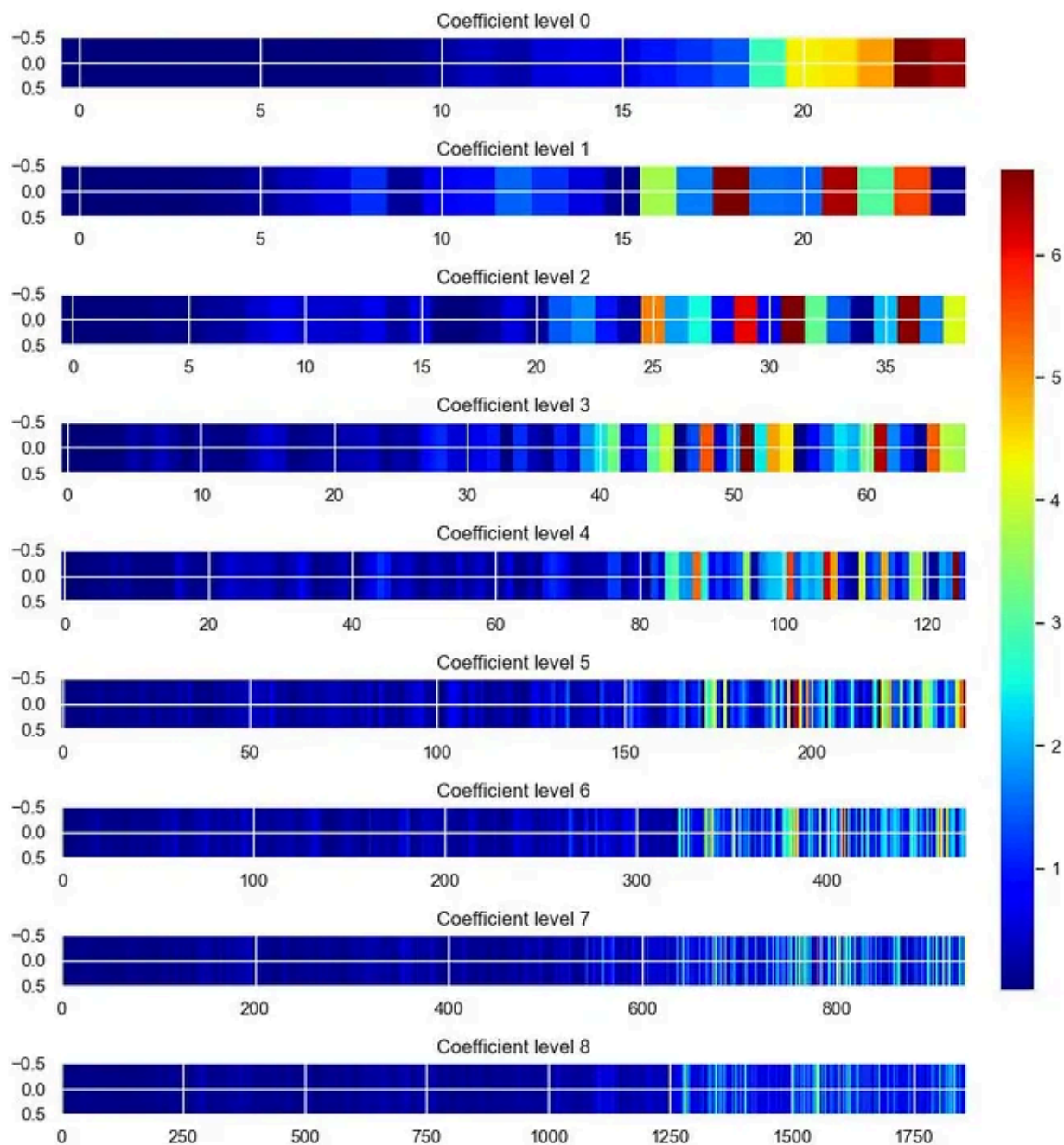
# Perform the Discrete Wavelet Transform with 'dbN'
wavelet = pywt.Wavelet('db6')
coeffs = pywt.wavedec(stock_data['Close'], wavelet, level=7)

# Plotting the Wavelet Coefficients using imshow
fig, axes = plt.subplots(len(coeffs), 1, figsize=(10, 10))

for i, ax in enumerate(axes):
    im = ax.imshow(np.abs(coeffs[i]).reshape(1, -1)), aspect='auto', cmap='jet')
    ax.set_title(f'Coefficient level {i}')
```

```
# Add a single colorbar on the far right
fig.subplots_adjust(right=0.85)
cbar_ax = fig.add_axes([0.86, 0.15, 0.03, 0.7])
fig.colorbar(im, cax=cbar_ax)

plt.tight_layout(rect=[0, 0, 0.85, 1])
plt.show()
```



The level in wavelet decomposition refers to the number of times the signal is decomposed into approximation (low-frequency) and detail (high-frequency) coefficients. As you go to higher levels of decomposition, the detail coefficients represent increasingly finer (higher frequency) details of the signal.

Best Wavelet Choices for Stock Price Analysis:

Daubechies Wavelets

- **Asymmetry:** Daubechies wavelets are not symmetric, which can be useful for capturing certain types of trends and abrupt changes in financial data.
- **Compact Support:** They have compact support, meaning they are localized in time, which helps in analyzing localized features in the data.
- **High Vanishing Moments:** Higher-order Daubechies wavelets (e.g., db4, db6) have more vanishing moments, which can be beneficial for detecting trends and smooth variations.

Symlets

- **Near Symmetry:** Symlets are nearly symmetric wavelets, proposed by Daubechies as modifications to the db family. This symmetry can be advantageous for certain applications where phase information is important.
- **Similar Properties:** Apart from symmetry, Symlets share many properties with Daubechies wavelets, such as compact support and high vanishing moments.

- **Smoothness:** Symlets tend to be smoother than Daubechies wavelets, which can be useful for capturing smooth trends in financial data.

For Trend Analysis

- **Daubechies Wavelets:** Might be preferred due to their ability to capture abrupt changes and trends more effectively.
- **Symlets:** Can also be used, especially if phase information and smoothness are important.

For Volatility Analysis

- **Symlets:** Their smoothness and symmetry might make them more suitable for capturing the volatility patterns in financial data.
- **Daubechies Wavelets:** Can still be used, especially if capturing abrupt changes in volatility is important.

Ultimately, the choice between Daubechies Wavelets and Symlets depends on the specific characteristics of your financial time series data and the goals of your analysis. It might be helpful to experiment with both and compare the results to see which wavelet performs better for your specific use case.

In wavelet analysis, the N in dbN (Daubechies wavelets) or $symN$ (Symlets) indicates the number of vanishing moments or, equivalently, the order of the wavelet. The number of vanishing moments affects smoothness of the wavelet — higher N means smoother wavelets.

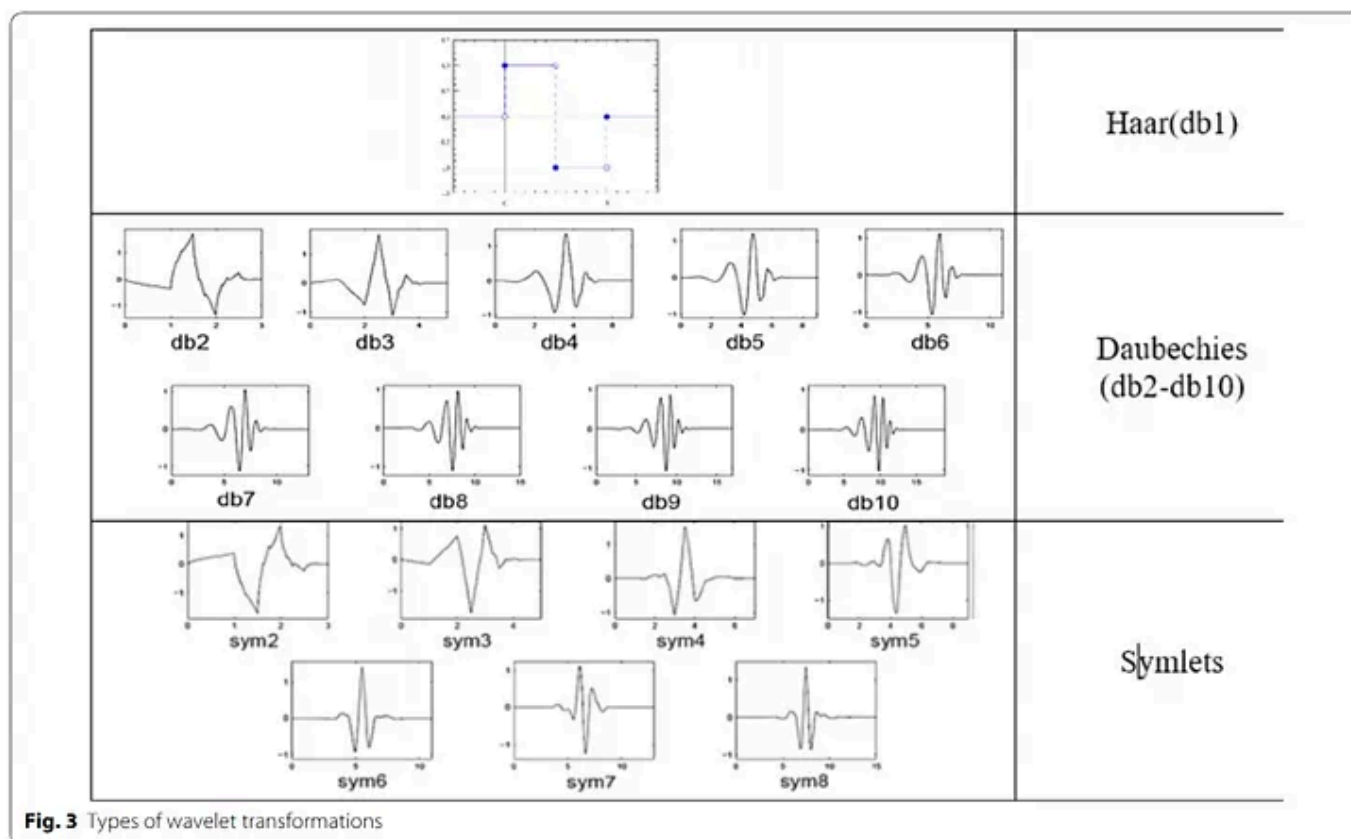


Image from [2]

Using Wavelet Transform, we decompose the 'Close' price and reconstruct it after removing the last 3 finer (higher frequency) details. We then feed this into the Long Short-Term Memory (LSTM) model.

According to the book [B1], it uses the Daubechies least asymmetric wavelet of length 8 LA(8) to analyze the relationship between the stock and the futures markets. PyWavelets (pywt) library does not directly have 'LA(8)'. However, this is effectively equivalent to using 'sym4' or 'sym6' (Symlets) in pywt, as Symlets are the least asymmetric variants of Daubechies wavelets.

I used Daubechies wavelets (db6) in my case.


```

# Load your stock data
df = stock_data.copy()

wavelet = pywt.Wavelet('db6')

# Apply wavelet transform
def wavelet_transform(data, lvl=8):
    coeff = pywt.wavedec(data, wavelet, mode='symmetric', level=lvl)
    return coeff

# Inverse wavelet transform
def inverse_wavelet_transform(coeff):
    # remove last 3 finer details
    for i in range(3):
        coeffs[i+5] = np.zeros(coeffs[i+5].shape)
    return pywt.waverec(coeff, wavelet, mode='symmetric')

# Prepare data
close_prices = df['Close'].values
wavelet_coeff = wavelet_transform(close_prices)
reconstructed_data = inverse_wavelet_transform(wavelet_coeff)

```

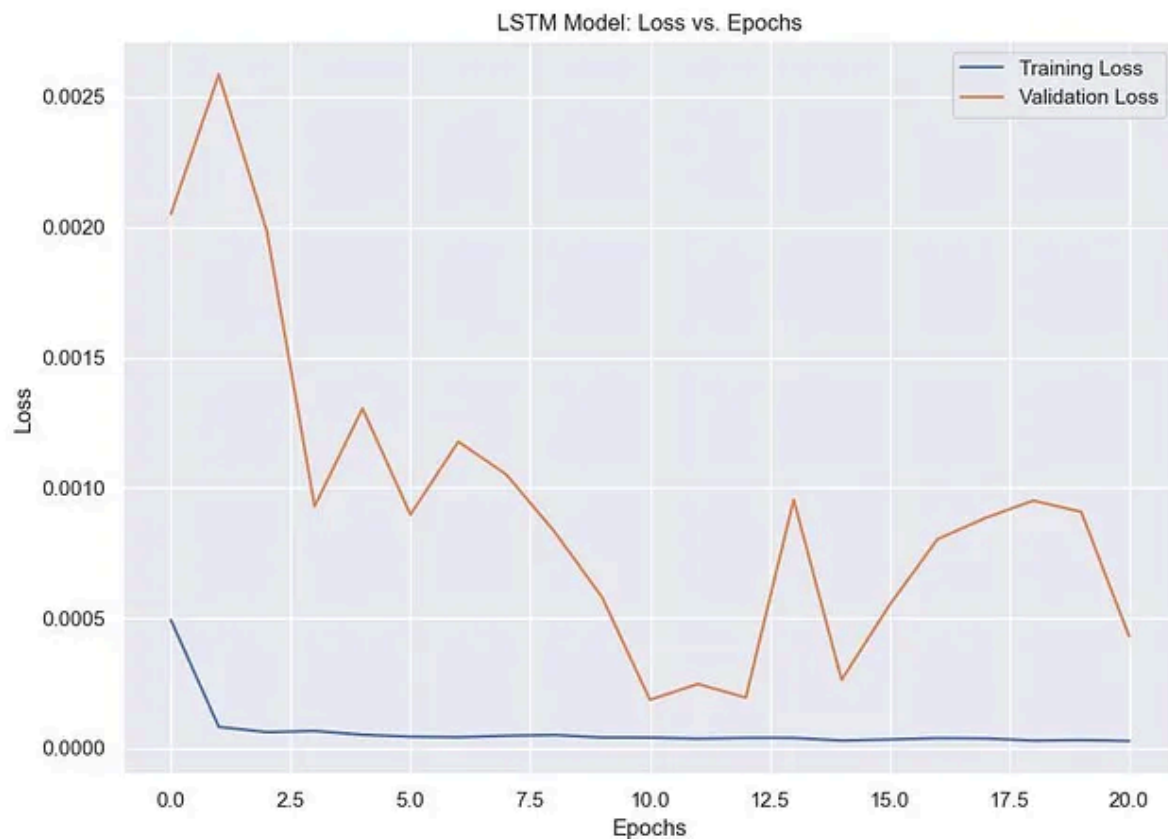
The LSTM model I used is Bi-Directional. It gives a 99.79% R^2 -score on the training set and a 96.14% R^2 -score on the testing set data.

```

# Build LSTM model
model = Sequential([
    Bidirectional(LSTM(units=256, return_sequences=True, input_shape=(seq_length,
    Dropout(0.3),
    Bidirectional(LSTM(units=128, return_sequences=False)),
    Dropout(0.3),
    RepeatVector(seq_length), # Number of future timesteps
    Dropout(0.2),
    Dense(units=64, activation='relu'),
    Dense(units=32),
    # Flatten layer to transition to Dense layer
    Flatten(),
    Dense(units=1)
])

```





	Metric	Train	Test
0	MAE	0.957407	3.654080
1	MSE	2.559654	21.505577
2	RMSE	1.599892	4.637411
3	MAPE	0.027133	0.021533
4	R2	0.997923	0.961410

Further Enhancement:

1. Explore other kinds of wavelets.
2. Fine-tune the model's hyperparameters.
3. Explore other deep learning models.

Recommended books:

[B1] An Introduction to Wavelet Theory in Finance: A Wavelet Multiscale Approach, Francis In, Sangbae Kim (2012, World Scientific Publishing Company)

[B2] Deep Learning for Time Series Data Cookbook, Vitor Cerqueira, Luís Roque (2024, Packt Publishing)

[B3] Stochastic Modeling: A Thorough Guide to Evaluate, Pre-Process, Model and Compare Time Series with MATLAB Software, Hossein Bonakdari, Mohammad Zeynoddin (2022, Elsevier)

[B4] Financial Machina: Machine Learning For Finance, Johann Strauss, Hayden Van Der Post, Vincent Bisette (2024, Reactive Publishing)

Reference:

[1] PHYSICS OF FLUIDS 18, 053302 (2006).

[2] BMC Medical Informatics and Decision Making 21(1), DOI:10.1186/s12911-021-01687-4, November 2021

Thank you for reading.



Written by Simon Leung

Edit profile

229 followers · 4 following

A fervent enthusiast, fueled by an insatiable curiosity for STEM (Science, Technology, Engineering, and Mathematics) disciplines.

